
Avance Rápido y *Backtracking*

Algoritmos y Estructuras de Datos II
Curso 2025/2026

- Adriana Piñero Iglesias
- Pablo Pérez Tudela

Índice

1. Lista de problemas resueltos	2
1.1. Problema asignado para Avance Rápido (AR)	2
1.2. Problema asignado para Backtracking (BT)	4
2. Resolución del problema por Avance Rápido	6
2.1. Diseño de la solución	6
2.2. Pseudocódigo del esquema voraz	6
2.3. Componentes y funciones auxiliares	6
2.4. Estudio teórico del tiempo de ejecución	8
2.5. Programación del algoritmo	9
2.6. Estudio experimental del tiempo de ejecución	10
2.7. Contraste teórico-experimental	12
3. Resolución del problema por Backtracking	13
3.1. Diseño	13
3.1.1. Representación del espacio de búsqueda	13
3.1.2. Estructuras de datos	13
3.1.3. Funciones auxiliares	13
3.1.4. Criterios de poda	14
3.1.5. Pseudocódigo del esquema completo	15
3.2. Estudio teórico del tiempo de ejecución	17
3.2.1. Solución sin poda	17
3.2.2. Solución con poda	18
3.3. Programación del algoritmo	20
3.4. Estudio experimental del tiempo de ejecución	21
3.5. Contraste teórico-experimental.	23
4. Instrucciones de compilación y ejecución	25
4.1. Avance Rápido	25
4.2. Backtracking	25
4.3. Gráficas adicionales	26
4.4. Compilación con make	26
5. Uso de la inteligencia artificial	27
6. Conclusiones	28
6.1. Valoraciones personales	28
6.2. Estimación de tiempo invertido	28

1. Lista de problemas resueltos

1.1. Problema asignado para Avance Rápido (AR)

Título: Organización en Clase (PID: 705)

Contexto

La asignación de los puestos a los alumnos en una clase es uno de los problemas más complicados a los que se enfrentan los profesores de secundaria, pues hay que tener en cuenta las relaciones personales de los alumnos (la amistad entre ellos) pero también la compenetración en el trabajo. En este problema se trata de agrupar los alumnos de dos en dos, pues suponemos que los pupitres son de dos plazas. No tendremos en cuenta la posición de los pupitres en la clase y supondremos que lo único que se pretende es agrupar a los alumnos de dos en dos (si el número es impar uno quedará sólo en un pupitre) intentando maximizar la amistad y al mismo tiempo la compenetración en el trabajo de los alumnos que sentamos juntos.

El Problema

Escribir un programa que realice una agrupación de alumnos de dos en dos maximizando la suma de los productos del grado de amistad y la compenetración en el trabajo de alumnos que se agrupan juntos. Se dispone para ello de una matriz de amistad y otra de trabajo, donde se guardan los grados de amistad y de compenetración, que pueden no ser recíprocos, por lo que las matrices no son simétricas. La amistad y la compenetración de un alumno consigo mismo se almacenará como cero. En cada pupitre se suman los grados de amistad de los alumnos en el pupitre, y se suman los grados de compenetración, y se multiplican los resultados de las dos sumas. El valor de beneficio es la suma de los productos obtenidos en todos los pupitres (sin incluir el posible pupitre donde sólo haya un alumno).

Suponer, por ejemplo, un caso donde el número de alumnos $N = 3$. Sean las matrices de amistad y trabajo las siguientes:

Amistad			Trabajo		
0	5	6	0	5	3
4	0	3	3	0	2
2	1	0	1	5	0

Si numeramos los alumnos con 0, 1 y 2, la agrupación $\langle 0, 1 \rangle$ tendrá valor 72 (que sale de: $(5 + 4) \cdot (5 + 3)$); la $\langle 0, 2 \rangle$ valor 32; y la $\langle 1, 2 \rangle$ valor 28, por lo que la mejor opción es sentar a los alumnos 0 y 1 juntos y dejar al 2 sólo.

Habrà que resolver el problema con avance rápido. La solución no tiene por qué ser la óptima, y se admitirán soluciones que no empeoren en más de un 30 % de las obtenidas con el programa de los profesores (en el ejemplo la única admisible será la de la agrupación $\langle 0, 1 \rangle$). Se considerará que el número máximo de alumnos es 100.

Entrada

La primera línea de la entrada contiene un entero, P , que indica el número de casos de prueba. Para cada caso de prueba, la primera línea contiene un entero, N , que indica el número de alumnos. A continuación, aparecen N líneas (una por alumno) en la entrada para la matriz de amistad, seguidas de otras N líneas (una por alumno) para la de trabajo. Cada línea contiene a su vez $N - 1$ enteros que representan los grados de amistad y compenetración del correspondiente

alumno con los restantes. Los valores en esas líneas estarán entre 1 y 9, y estarán separados por espacios en blanco. Obsérvese que se omiten los valores 0 de cada alumno consigo mismo.

Salida

La salida constará de $2 \cdot P$ líneas. Las líneas impares contendrán el valor de beneficio total obtenido para cada uno de los casos de prueba. Las líneas pares contendrán una secuencia de N valores, uno por cada alumno de cada caso de prueba. La secuencia indica los emparejamientos realizados: los valores en posiciones impares corresponden al primer alumno de cada pareja, y los de posiciones pares al segundo. Si N es impar, el último alumno queda desparejado.

Registro de envíos al juez on-line

#	Tiempo de Concurso ▾	País	Equipo	Problema	Lenguaje	Resultado	Estado	🔍
123	1041:46:47		g9_g9 29	I_AR	C++	3 Accepted	final	
122	1041:42:58		g9_g9 29	I_AR	C++	0 Compile Time Error	final	
121	1038:53:26		g9_g9 29	I_AR	C++	3 Accepted	final	
120	1038:51:54		g9_g9 29	I_AR	C++	0 Compile Time Error	final	

Figura 1: Envíos realizados a través de Mooshak.

- **Envío 120. Compile Time Error:** se nos olvidó hacer el *casting* del valor devuelto por `S.size()` (que es `unsigned int`) al comparar con $N - 1$ dentro de la función `Solucion`.
- **Envío 121. Accepted:** tras corregir ese detalle, el programa pasó a tiempo en Mooshak.
- **Envío 122. Compile Time Error:** reincidimos en el mismo fallo de *casting* en otro punto del código.
- **Envío 123. Accepted:** entrega definitiva. En lugar de borrar el candidato seleccionado de la lista C (operación con coste lineal en el tamaño de C), llevamos un índice entero `idx` que avanza una posición en cada iteración. Es un cambio pequeño pero, según las medidas experimentales, evita un coste adicional notable en cada paso del bucle `voraz`.

1.2. Problema asignado para Backtracking (BT)

Título: Nos Vamos de Boda (PID: 901)

Contexto

Uno de nuestros mejores amigos se casa y estamos emocionados porque es el primero del grupo que lo hace. De hecho, es la primera vez que asistimos a una boda y para proveernos del vestuario y complementos necesarios nos vamos a El Recorte Británico, donde disponen de un surtido variado. Necesitaremos una camisa, un cinturón, unos zapatos, unos gemelos, una corbata, etcétera.

El problema

Tenemos que comprar un modelo de cada tipo de prenda, y sólo uno. Disponemos de un presupuesto limitado, que no podemos rebasar, aunque nuestra pretensión es gastar lo máximo posible dentro de ese presupuesto. De cada tipo de prenda nos pueden ofrecer diferentes modelos, cada uno con su precio. Existe la posibilidad de que nuestro presupuesto no nos permitiera comprar un modelo de cada tipo de prenda.

Entrada

La primera línea de la entrada contendrá un número N , que indicará el número de casos de prueba. Para cada caso de prueba, la primera línea contendrá dos enteros, M y C , $1 \leq M \leq 200$, y $1 \leq C \leq 20$, separados por un espacio en blanco, donde M es el presupuesto disponible y C es el número de prendas a comprar. A continuación siguen C líneas, una por prenda, cada una de ellas constará de un primer entero K , $1 \leq K \leq 20$, que indica la cantidad de modelos distintos de cada prenda, seguido de K enteros con valores comprendidos entre 1 y 200, que indicarán los precios de los K modelos de dicha prenda; los valores de cada línea irán separados por un espacio en blanco.

Salida

Para cada caso de prueba la salida constará de una línea con un entero que indicará la mayor cantidad de dinero necesario para adquirir un modelo de cada tipo de prenda sin sobrepasar el presupuesto. En caso de no haber solución, la línea contendrá la cadena “no solution”.

Ejemplo:

- Entrada: $N = 1$, $M = 100$, $C = 4$. Conjuntos de precios P_i con cardinalidad K_i :
 $P_1 (K_1 = 3) = \{8, 6, 4\}$
 $P_2 (K_2 = 2) = \{5, 10\}$
 $P_3 (K_3 = 4) = \{1, 3, 3, 7\}$
 $P_4 (K_4 = 4) = \{50, 14, 23, 8\}$
- Salida: 75
- Explicación: la suma de los precios máximos de cada conjunto es $8 + 10 + 7 + 50 = 75$. Como $75 \leq 100$, ya es una solución admisible y, además, no se puede mejorar (es la cota superior). El algoritmo la encuentra en la primera rama y descarta el resto por la poda de optimalidad. El vector de precios seleccionados es $p = (8, 10, 7, 50)$.

Registro de envíos al juez on-line

#	Tempo de Concurso ▾	Pais	Equipo	Problema	Lenguaje	Resultado	Estado	5
53	370:16:34		g9_g9 29	F_Ba	C++	3 Accepted	final	
45	303:02:15		g9_g9 29	F_Ba	C++	0 Time Limit Exceeded	final	
44	296:03:07		g9_g9 29	F_Ba	C++	2 Time Limit Exceeded	final	
43	296:01:57		g9_g9 29	F_Ba	C++	0 Compile Time Error	final	
42	294:54:00		g9_g9 29	F_Ba	C++	0 Time Limit Exceeded	final	
41	294:52:59		g9_g9 29	F_Ba	C++	0 Compile Time Error	final	

Figura 2: Envíos realizados a través de Mooshak.

- **Envío 41. Compile Time Error:** primera versión, implementada con un DFS recursivo. Aunque la idea era correcta, decidimos rehacerla siguiendo el esquema iterativo de *backtracking* visto en clase. El error venía de un *casting* olvidado al comparar tamaños.
- **Envío 42. Time Limit Exceeded:** ya compila, pero al no incorporar ninguna poda recorre todo el árbol y excede el límite de tiempo en cuanto crecen los casos de prueba.
- **Envío 43. Compile Time Error:** añadimos dos podas iniciales (presupuesto y cota mínima). Fallo de compilación porque una variable del bucle de casos quedaba sin inicializar.
- **Envío 44. Time Limit Exceeded:** las dos podas no eran suficientes; el árbol todavía crecía demasiado.
- **Envío 45. Time Limit Exceeded:** reorganizamos el código en funciones auxiliares para que se pareciera al esquema teórico, pero como las condiciones de poda eran las mismas, el tiempo no mejoró.
- **Envío 63. Accepted:** añadimos las dos podas que faltaban (la cota por máximos y, sobre todo, la matriz de visitados $V[nivel][suma]$). Con ellas el número de nodos explorados cae drásticamente y el problema entra en tiempo.

2. Resolución del problema por Avance Rápido

2.1. Diseño de la solución

Seguimos el esquema general de un voraz iterativo: en cada paso elegimos la pareja de alumnos con mayor beneficio aún disponible, comprobando que ninguno de los dos esté ya sentado. Como hay que decidir sobre todos los pares posibles, antes de empezar generamos la lista completa de candidatos y la ordenamos por beneficio decreciente.

2.2. Pseudocódigo del esquema voraz

El siguiente pseudocódigo (Algoritmo 1) representa la solución propuesta:

Algoritmo 1 Esquema voraz de agrupamiento optimizado

Entrada: Matrices A (amistad) y T (trabajo) de dimensión $N \times N$, entero N

Salida: Conjunto S de pupitres y beneficio total b

```
1: function VORAZ( $A, T, N$ )
2:    $S \leftarrow \emptyset$ 
3:    $asignado \leftarrow$  vector de  $N$  booleanos a FALSO
4:    $C \leftarrow$  GENERARCANDIDATOS( $A, T, N$ )
5:    $b \leftarrow 0$ ;  $idx \leftarrow 0$ 
6:   while  $idx < |C|$  y no SOLUCION( $S, N$ ) do
7:      $x \leftarrow$  SELECCIONAR( $C, idx$ )
8:      $idx \leftarrow idx + 1$  ▷ equivale a  $C \leftarrow C - \{x\}$ 
9:     if FACTIBLE( $asignado, x$ ) then
10:      INSERTAR( $S, x$ )
11:       $asignado[x.u] \leftarrow$  VERDADERO
12:       $asignado[x.v] \leftarrow$  VERDADERO
13:       $b \leftarrow b + x.beneficio$ 
14:     end if
15:   end while
16:   if  $S = \emptyset \wedge N > 1$  then devolver “no hay solución”
17:   else devolver ( $S, b$ )
18:   end if
19: end function
```

2.3. Componentes y funciones auxiliares

Las componentes principales son:

- **Solución S :** conjunto de los pares de alumnos ya asignados a un pupitre.
- **Candidatos C :** lista de las $\binom{N}{2}$ parejas posibles, calculada al principio y ordenada por beneficio decreciente.
- **Vector asignado:** vector booleano de N posiciones que indica en $\mathcal{O}(1)$ si un alumno ya está sentado. Sustituye al borrado físico de elementos en C (función `modificarC` del esquema general), que tendría coste lineal en cada paso.
- **Índice idx :** variable entera que avanza una posición por iteración. Equivale a la operación $C \leftarrow C - \{x\}$ pero en tiempo constante.

Funciones auxiliares:

- **GenerarCandidatos:** agrupa todas las posibles parejas y calcula su beneficio según la fórmula $(A[i][j] + A[j][i]) \times (T[i][j] + T[j][i])$, devolviendo la lista ordenada por beneficio decreciente (con desempate por índices).
- **Seleccionar:** devuelve el siguiente candidato más prometedor de la lista ordenada en $\mathcal{O}(1)$.
- **Factible:** comprueba que ambos alumnos del candidato x no hayan sido sentados aún.
- **Insertar:** añade la pareja a la solución parcial.
- **Solucion:** determina si ya hemos alcanzado un emparejamiento completo. Consideramos que tenemos solución cuando se han sentado N alumnos (si N es par) o $N - 1$ alumnos (si N es impar), es decir, cuando $2 \cdot |S| \geq N - 1$.

El Algoritmo 2 recoge la implementación de las funciones auxiliares.

Algoritmo 2 Funciones auxiliares del esquema voraz

```
1: function GENERARCANDIDATOS( $A, T, N$ )
2:    $C \leftarrow \emptyset$ 
3:   for  $i \leftarrow 0$  hasta  $N - 1$  do
4:     for  $j \leftarrow i + 1$  hasta  $N - 1$  do
5:        $b \leftarrow (A[i, j] + A[j, i]) \cdot (T[i, j] + T[j, i])$ 
6:        $C \leftarrow C \cup \{(i, j, b)\}$ 
7:     end for
8:   end for
9:   Ordenar  $C$  por beneficio decreciente (desempate por índices)
10:  devolver  $C$ 
11: end function

12: function SELECCIONAR( $C, idx$ )
13:  devolver  $C[idx]$ 
14: end function

15: function FACTIBLE( $asignado, x$ )
16:  devolver  $\neg asignado[x.u] \wedge \neg asignado[x.v]$ 
17: end function

18: function INSERTAR( $S, x$ )
19:   $S \leftarrow S \cup \{x\}$ 
20: end function

21: function SOLUCION( $S, N$ )
22:  devolver  $(2 \cdot |S| \geq N - 1)$ 
23: end function
```

2.4. Estudio teórico del tiempo de ejecución

Sea n el número de alumnos y $k = \frac{n(n-1)}{2}$ el número total de parejas candidatas. El tiempo del peor caso del algoritmo viene dado, según el modelo RAM, por la siguiente expresión:

$$\begin{aligned} t_M(n) = & c_0n + c_1 + c_2(n+1) + c_3 \frac{n(n+1)}{2} + c_4 \frac{n(n-1)}{2} \\ & + c_5(k \log k) \\ & + c_6(k+1) + c_7k \\ & + c_8k \\ & + c_9k \\ & + c_{10} \left(\frac{n}{2} \right) \end{aligned}$$

Constantes:

- c_0 : Inicialización del vector de booleanos **asignado**.
- $c_1 \dots c_4$: Inicialización de variables y coste de los bucles anidados para generar candidatos.
- c_5 : Coste del algoritmo **sort** sobre los k candidatos.
- c_6, c_7, c_9 : Condición del **while**, extracción mediante el índice (**seleccionar**) y comprobación de viabilidad (**factible**).
- c_8 : Incremento secuencial del índice lógico (**idx++**) en cada una de las k iteraciones.
- c_{10} : Operaciones dentro de la condición de factibilidad, que como máximo ocurren $n/2$ veces (llamada a **insertar** en S , actualización de las dos posiciones de **asignado** y suma del **beneficioTotal**).

Sabiendo que el coste de ordenación se puede desarrollar como $k \log k \approx \frac{n^2}{2} \log \left(\frac{n^2}{2} \right) = \frac{n^2}{2} (2 \log n - \log 2) \in \Theta(n^2 \log n)$, y sustituyendo $k \approx \frac{n^2}{2}$ en el resto de la ecuación, agrupamos términos. Como el término de mayor orden es $n^2 \log n$, obtenemos:

$$t_M(n) = an^2 \log n + bn^2 + cn + d \in \Theta(n^2 \log n)$$

El mejor caso se daría cuando las primeras $n/2$ parejas de la lista ordenada son completamente disjuntas (ningún alumno se repite), provocando que la condición de parada del bucle (**Solucion(S, N)**) se alcance y el algoritmo termine tras $n/2$ iteraciones:

$$\begin{aligned} t_m(n) = & c_0n + c_1 + c_2(n+1) + c_3 \frac{n(n+1)}{2} + c_4 \frac{n(n-1)}{2} \\ & + c_5(k \log k) \\ & + c_6 \left(\frac{n}{2} + 1 \right) + c_7 \left(\frac{n}{2} \right) \\ & + c_8 \left(\frac{n}{2} \right) \\ & + c_9 \left(\frac{n}{2} \right) \\ & + c_{10} \left(\frac{n}{2} \right) \end{aligned}$$

Agrupando nuevamente los términos, el mejor caso quedaría como:

$$t_m(n) = a'n^2 \log n + b'n^2 + c'n + d' \in \Theta(n^2 \log n)$$

Por tanto, el algoritmo tiene orden exacto $\Theta(n^2 \log n)$ tanto en el peor caso como en el mejor caso, ya que el coste de la fase de ordenación domina sobre el resto.

2.5. Programación del algoritmo

Hemos repartido la implementación en los siguientes ficheros:

- `solucionAR.cpp` y `solucionAR.h`: implementación del voraz, junto con la lectura del fichero de casos y el formato de salida. Es la versión que enviamos a Mooshak. La cabecera expone los *structs* `Pareja` y `ResultadoVoraz` y las funciones del esquema (`generarCandidatos`, `Solucion`, `seleccionar`, `factible`, `insertar`, `voraz`) para reutilizarlas en los tests y en la medición de tiempos.
- `tiempos_AR.cpp`: genera entradas aleatorias de varios tamaños N (espaciado multiplicativo) y mide la mediana del tiempo de `voraz` sobre 51 repeticiones por tamaño. Vuelca los resultados a un CSV.
- `tests_unitarios_AR.cpp`: tests que comprueban casos sencillos (el ejemplo del enunciado, $N = 2$, $N = 4$ con parejas disjuntas) y verifican que ningún alumno aparezca dos veces en la salida.
- `regresion_AR.py`: lee el CSV, ajusta el modelo $t \approx a n^2 \log_2 n + b$ y dibuja la gráfica con el valor de R^2 .
- `Makefile`: compila las soluciones (`solucionAR`, `solucionBT` y `solucionBT_noPoda`) con las opciones `-O2 -std=c++17 -Wall`.

2.6. Estudio experimental del tiempo de ejecución

Entorno de medición. Para cada tamaño N medimos el tiempo de `solucionAR` (sin la lectura) sobre 51 instancias aleatorias y nos quedamos con la mediana, que es más robusta frente a picos puntuales del sistema operativo.

Tipos de instancias. Usamos tres generadores para no depender de una sola distribución:

- *Uniforme*: valores enteros entre 1 y 9 al azar.
- *Adversaria*: solo 1 ó 9, lo que genera muchos empates al ordenar.
- *Correlacionada*: amistad y trabajo correlacionados, más cercano a un caso real.

Tamaños probados. 19 valores con espaciado multiplicativo (razón $\approx 1,5$):

$N \in \{4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384, 512, 768, 1024, 1536, 2048\}$. La tabla siguiente recoge los tiempos y el factor $t(2N)/t(N)$ para los pares de potencias de 2:

N	t (ms)	$t(2N)/t(N)$	N	t (ms)	$t(2N)/t(N)$
4	0.000200	—	128	0.532800	4,57
6	0.000500	—	192	1.310400	—
8	0.000900	4,50	256	2.431300	4,56
12	0.002300	—	384	5.765400	—
16	0.004900	5,44	512	10.736400	4,42
24	0.013600	—	768	24.289600	—
32	0.024600	5,02	1024	62.680500	5,84
48	0.057400	—	1536	152.908800	—
64	0.116500	4,73	2048	254.129700	4,05
96	0.290900	—			

La columna $t(2N)/t(N)$ es el factor por el que se multiplica el tiempo al duplicar N . La cota $\Theta(n^2 \log n)$ predice

$$\frac{(2n)^2 \log(2n)}{n^2 \log n} = 4 \left(1 + \frac{1}{\log n} \right),$$

que para los n de la tabla va de $\approx 6,0$ (en $n = 4$) a $\approx 4,4$ (en $n = 512$). Los valores experimentales caen dentro de ese rango, salvo el cociente 5,84 en la transición $512 \rightarrow 1024$, que atribuimos a efectos de caché al ordenar los $\approx 523\,000$ candidatos. La Figura 3 muestra los tiempos en escala lineal. En escala log-log (Figura 4) la pendiente experimental es 2,255, coherente con la cota, y para $n^2 \log n$ la pendiente esperada está algo por encima de 2 (el factor $\log n$ la sube ligeramente en este rango).

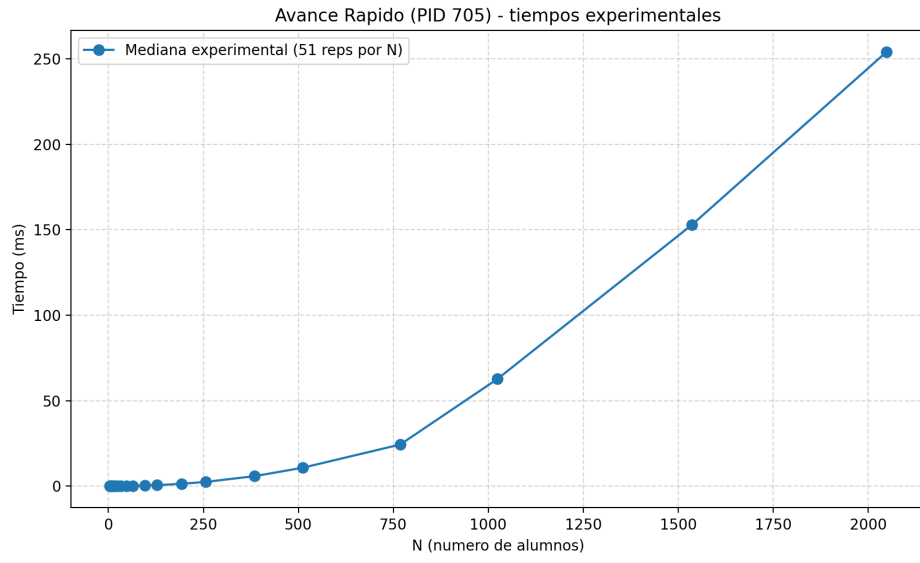


Figura 3: Tiempos experimentales del algoritmo voraz en escala lineal.

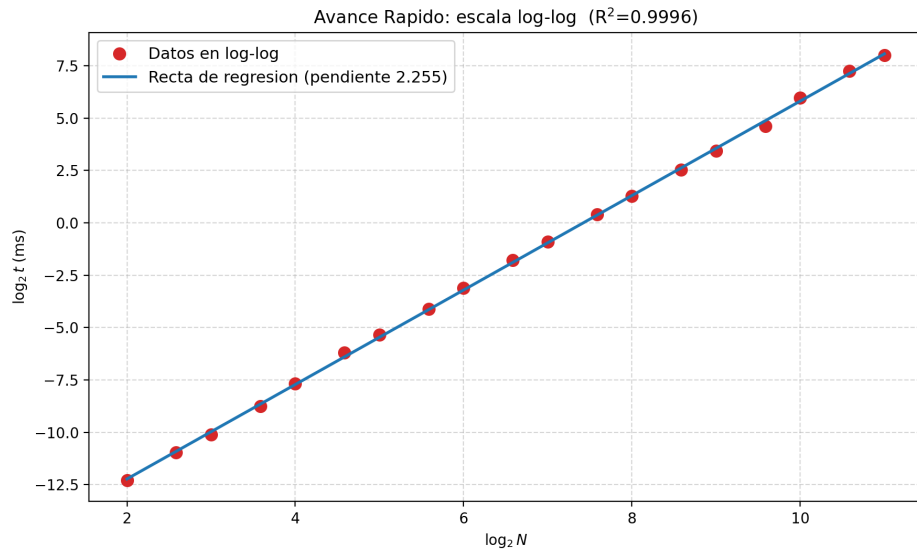


Figura 4: Mismos tiempos en escala $\log_2$ - $\log_2$. La pendiente del ajuste lineal es $\approx 2,255$, con $R^2 = 0,9996$.

2.7. Contraste teórico-experimental

Con el script `regresion_AR.py` ajustamos los 19 puntos al modelo

$$t(n) \approx a \cdot n^2 \log_2 n + b,$$

y obtenemos:

- $a \approx 5,65 \times 10^{-6}$ ms.
- $b \approx -2,01 \times 10^{-1}$ ms.
- $R^2 = 0,9967$.

Un R^2 tan cercano a 1 confirma que el comportamiento experimental encaja con la cota $\Theta(n^2 \log n)$, dominada por la ordenación de los $k = n(n-1)/2$ candidatos. El término b sale ligeramente negativo, pero su valor absoluto es del orden de la precisión del reloj para N pequeño, así que no afecta al ajuste global. La Figura 5 muestra los puntos y la curva ajustada.

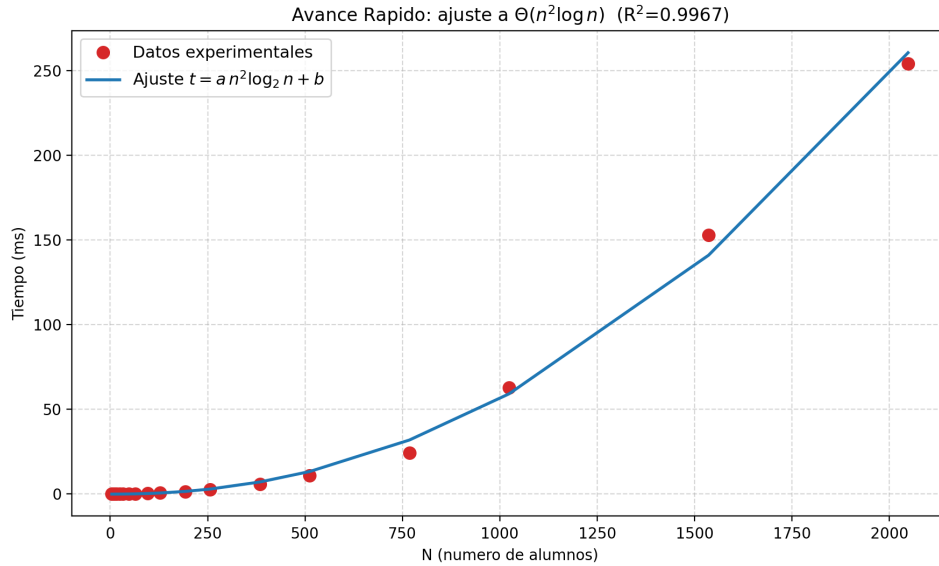


Figura 5: Tiempos experimentales y ajuste $t \approx a n^2 \log_2 n + b$ del algoritmo voraz (problema 705). $R^2 = 0,9967$.

3. Resolución del problema por Backtracking

3.1. Diseño

A continuación explicamos los distintos elementos de la solución.

3.1.1. Representación del espacio de búsqueda

El problema consiste en maximizar el dinero gastado sin pasarse del presupuesto, eligiendo un modelo por cada tipo de prenda. Lo abordamos con un esquema de *backtracking* iterativo de maximización (Algoritmo 7). Para no recalcular el coste acumulado en cada nodo usamos un vector `suma` que se actualiza de forma incremental, manteniendo el coste por nodo en $\mathcal{O}(1)$.

El espacio de estados forma un árbol k -ario en el que el nodo de profundidad i representa la elección de un modelo de la prenda i -ésima. Una solución es una tupla

$$X = (x_0, x_1, \dots, x_{N-1}),$$

donde N es el número de prendas y cada $x_i \in \{0, 1, \dots, K_i - 1\}$ es el índice del modelo escogido para la prenda i (siendo K_i el número de modelos disponibles).

3.1.2. Estructuras de datos

Las estructuras globales que necesita el algoritmo son las siguientes:

- **Matriz de catálogo (C):** matriz bidimensional con los precios de los modelos. Antes de ejecutar el *backtracking* ordenamos cada fila de mayor a menor; así el algoritmo prueba primero las ramas con modelos más caros y es más probable que actualice `solucionOptima` pronto, lo que hace más eficaz la poda por optimalidad.
- **Vectores de cotas m y M :** dos vectores precomputados de tamaño $N + 1$ con la suma de los precios mínimos y máximos de las prendas que aún no se han fijado. Para un nivel i :

$$m[i] = \sum_{j=i}^{N-1} \min(C[j]), \quad M[i] = \sum_{j=i}^{N-1} \max(C[j]).$$

Los usamos en los criterios de poda. La precomputación se describe en el Algoritmo 6.

- **Matriz de visitados (V):** matriz booleana $N \times (P + 1)$, donde P es el presupuesto. Si $V[i][s]$ es cierto, significa que ya hemos explorado a fondo el subárbol que parte del nivel i con suma acumulada s y no encontramos nada mejor, por lo que podemos podar. Esta poda por memoización ha sido la decisiva para que la solución entrara en tiempo en Mooshak.

El Algoritmo 3 resume estas estructuras.

3.1.3. Funciones auxiliares

Hemos implementado el *backtracking* de forma iterativa, evitando la sobrecarga de la pila de llamadas. El estado del recorrido se mantiene en los vectores `solucionAct` (índice elegido en cada nivel) y `suma` (coste acumulado). El Algoritmo 4 contiene el pseudocódigo de las siguientes funciones auxiliares:

- **generar:** pasa al siguiente hermano del nodo actual. Como las filas del catálogo están ordenadas de mayor a menor, esto equivale a probar los modelos en orden de precio decreciente.

- **retroceder**: sube un nivel en el árbol; deja la variable de decisión a -1 y decrementa nivel.
- **actualizarSuma**: actualiza el coste acumulado en $\mathcal{O}(1)$ usando el del nivel anterior:

$$S_{nivel} = S_{nivel-1} + C[nivel][x_{nivel}].$$

- **solucion**: comprueba que el estado actual sea una solución completa y válida:

$$(nivel = N - 1) \wedge (S_{nivel} \leq P).$$

3.1.4. Criterios de poda

Las podas son la parte clave de la solución, ya que sin ellas el árbol crece demasiado. Todas se concentran en la función **criterio**, que aplica cuatro condiciones; basta que falle una para que el subárbol no se explore. Sea S_{nivel} el coste acumulado en el estado actual:

1. Si $S_{nivel} > P$, ya nos hemos pasado del presupuesto.
2. Si $S_{nivel} + m[nivel + 1] > P$, ni siquiera escogiendo los modelos más baratos en el resto de prendas cabríamos en el presupuesto.
3. Si $S_{nivel} + M[nivel + 1] \leq solucionOptima$, ni escogiendo los modelos más caros mejoráramos la mejor solución encontrada hasta ahora.
4. Si $V[nivel][S_{nivel}]$ es cierto, ya hemos pasado por este mismo estado en otra rama y sabemos que no aporta nada nuevo.

El Algoritmo 5 muestra estas comprobaciones en pseudocódigo.

3.1.5. Pseudocódigo del esquema completo

Algoritmo 3 Estructuras globales de datos

```
1: Variables Globales:
2:   Entero  $P$                                 ▷ Presupuesto máximo
3:   Entero  $N$                                 ▷ Número total de prendas
4:   Matriz de Enteros  $C_{N \times K_i}$           ▷ Catálogo ordenado decrecientemente
5:   Vector de Enteros  $m_{N+1}$               ▷ Sumas de los mínimos elementos restantes
6:   Vector de Enteros  $M_{N+1}$               ▷ Sumas de los máximos elementos restantes
7:   Matriz de Booleanos  $V_{N \times (P+1)}$     ▷ Matriz de estados visitados
```

Algoritmo 4 Funciones auxiliares del esquema de *backtracking*

```
1: function GENERAR( $nivel$ ,  $solucionAct$ )
2:    $solucionAct[nivel] \leftarrow solucionAct[nivel] + 1$ 
3: end function

4: function RETROCEDER(ref  $nivel$ , ref  $s$ )
5:    $s[nivel] \leftarrow -1$ 
6:    $nivel \leftarrow nivel - 1$ 
7: end function

8: function ACTUALIZARSUMA( $nivel$ ,  $solucionAct$ , ref  $suma$ )
9:   if  $nivel > 0$  then
10:     $sumaPrevia \leftarrow suma[nivel - 1]$ 
11:   else
12:     $sumaPrevia \leftarrow 0$ 
13:   end if
14:    $suma[nivel] \leftarrow sumaPrevia + C[nivel][solucionAct[nivel]]$ 
15: end function

16: function SOLUCION( $nivel$ ,  $sumaNivel$ )
17:   devolver ( $nivel == N - 1$ )  $\wedge$  ( $sumaNivel \leq P$ )
18: end function
```

Algoritmo 5 Criterios de poda

```
1: function CRITERIO(nivel, sumaNivel, solucionOptima)
2:   if sumaNivel > P then devolver Falso
3:   end if
4:   if nivel  $\geq N - 1$  then devolver Falso
5:   end if
6:   if sumaNivel + m[nivel + 1] > P then devolver Falso
7:   end if
8:   if sumaNivel + M[nivel + 1]  $\leq$  solucionOptima then devolver Falso
9:   end if
10:  if V[nivel][sumaNivel] then devolver Falso
11:  end if
12:  devolver Verdadero
13: end function
```

Algoritmo 6 Precomputación de vectores auxiliares

```
1: function VECTORES_AUXILIARES
2:   m  $\leftarrow$  Vector de  $N + 1$  ceros
3:   M  $\leftarrow$  Vector de  $N + 1$  ceros
4:   for i  $\leftarrow N - 1$  downto 0 do
5:     m[i]  $\leftarrow$  m[i + 1] + último elemento de C[i]
6:     M[i]  $\leftarrow$  M[i + 1] + primer elemento de C[i]
7:   end for
8: end function
```

Algoritmo 7 Esquema iterativo de *backtracking*

```
1: function BACKTRACKING
2:   solucionAct  $\leftarrow$  Vector de  $N$  elementos inicializados a  $-1$ 
3:   suma  $\leftarrow$  Vector de  $N$  elementos inicializados a  $0$ 
4:    $V \leftarrow$  Matriz de  $N \times (P + 1)$  inicializada a Falso
5:   nivel  $\leftarrow 0$ 
6:   solucionOptima  $\leftarrow -1$ 
7:   while nivel  $\neq -1$  do
8:     GENERAR(nivel, solucionAct)
9:     if solucionAct[nivel]  $\geq$  tamaño de  $C$ [nivel] then
10:      RETROCEDER(nivel, solucionAct)
11:      continue
12:     end if
13:     ACTUALIZARSUMA(nivel, solucionAct, suma)
14:     if suma[nivel]  $> P$  then
15:       continue
16:     end if
17:     if SOLUCION(nivel, suma[nivel]) then
18:       if suma[nivel]  $>$  solucionOptima then
19:         solucionOptima  $\leftarrow$  suma[nivel]
20:       end if
21:       continue
22:     end if
23:     if CRITERIO(nivel, suma[nivel], solucionOptima) then
24:        $V$ [nivel][suma[nivel]]  $\leftarrow$  Verdadero
25:       nivel  $\leftarrow$  nivel + 1
26:       solucionAct[nivel]  $\leftarrow -1$ 
27:     end if
28:   end while
29:   devolver solucionOptima
30: end function
```

3.2. Estudio teórico del tiempo de ejecución

Para el análisis usamos las siguientes variables:

- $n = \text{numPrendas}$.
- $K = \max_{0 \leq i < n} |\text{catalogo}[i]|$.
- $M = \text{presupuesto}$.

3.2.1. Solución sin poda

El tiempo de ejecución en el peor caso $t_M(n)$ se obtiene sumando el coste de la precomputación y el producto del número de nodos del árbol por el coste por nodo:

- **Nodos:** sin podas, el algoritmo recorre todo el árbol K -ario de profundidad n , así que el número de nodos es $\mathcal{O}(K^n)$.
- **Tiempo por nodo:** las funciones auxiliares trabajan en $\Theta(1)$.

- **Precomputación:** `leerArbol` ordena cada una de las n filas del catálogo (de tamaño máximo K), con coste $\mathcal{O}(n \cdot K \log K)$. La inicialización de los vectores `solucionAct` y `suma` cuesta $\Theta(n)$.

Por tanto:

$$t_M(n, K) \in \mathcal{O}(n \cdot K \log K) + \mathcal{O}(K^n) \cdot \Theta(1) = \mathcal{O}(K^n).$$

El término exponencial domina sobre el de precomputación.

El mejor caso $t_m(n, K)$ se da cuando todos los modelos de la primera prenda superan ya el presupuesto: el algoritmo recorre los K_0 candidatos del primer nivel, los descarta en $\Theta(1)$ y termina sin entrar a profundidad mayor. Aun así, el coste no puede ser inferior al de la precomputación, así que:

$$t_m(n, K) \in \Omega(n \cdot K \log K).$$

3.2.2. Solución con poda

Con las cuatro podas, dos de ellas son las que dominan la complejidad asintótica:

- **Poda por cotas precomputadas** (vectores `minimos` y `maximos`): descarta ramas que ni con los precios más caros restantes mejorarían `solucionOptima`, o que ni con los más baratos cabrían dentro del presupuesto.
- **Poda por memoización** (matriz `visitado`): cuando entramos por primera vez en un estado $(nivel, suma)$, marcamos `visitado[nivel][suma] = true`. Si otra rama vuelve a llegar a ese mismo par, la poda 4 lo corta sin generar hijos.

Conteo del número de nodos visitados. La memoización es la clave del análisis: como solo exploramos los hijos de un estado $(nivel, suma)$ la primera vez que lo encontramos, el número de *estados padres* explorados es, a lo sumo, el de pares $(nivel, suma)$ distintos. Hay n niveles y la suma puede tomar valores enteros entre 0 y M , así que hay como mucho

$$\mathcal{O}(n \cdot M)$$

estados padres distintos. De cada uno generamos hasta K hijos, y la función `criterio` los evalúa en $\Theta(1)$. Por tanto, el número total de nodos visitados (incluidos los podados) está acotado por

$$\mathcal{O}(n \cdot M \cdot K).$$

Costes de precomputación. Antes del bucle hay que:

- Ordenar cada fila del catálogo: $\mathcal{O}(n \cdot K \log K)$.
- Calcular `vectoresAuxiliares` (m y M): $\Theta(n)$.
- Inicializar la matriz `visitado` a falso: $\Theta(n \cdot M)$.

Peor caso. Sumando precomputación y exploración:

$$t_M(n, M, K) \in \mathcal{O}(n \cdot K \log K + n \cdot M \cdot K).$$

Como $K \log K$ es despreciable frente a $M \cdot K$ cuando M es grande, esta cota se simplifica en la práctica a $\mathcal{O}(n \cdot M \cdot K)$. Las podas hacen que el número real de nodos visitados sea bastante menor que el máximo teórico.

Mejor caso. Se da cuando la primera rama explorada ya alcanza el valor óptimo (por ejemplo, cuando la suma de los precios máximos es $\leq P$, como en el ejemplo del enunciado): a partir de ahí, la poda 3 (**maximos**) corta todas las ramas siguientes en $\mathcal{O}(1)$ cada una. El coste lo determina entonces la precomputación:

$$t_m(n, M, K) \in \Omega(n \cdot K \log K + n \cdot M).$$

3.3. Programación del algoritmo

Hemos repartido la implementación en los siguientes ficheros:

- `solucionBacktracking.cpp` y `solucionBacktracking.h`: implementación con podas, la que enviamos y fue aceptada en Mooshak.
- `solucionBacktracking_noPoda.cpp` y `solucionBacktracking_noPoda.h`: misma implementación pero sin las podas; la usamos como referencia en el estudio experimental.
- `tests_unitarios_BT.cpp`: tests que verifican la corrección sobre los casos del enunciado y casos límite (presupuesto insuficiente, una sola prenda, etc.).
- `tiempos_BT.cpp`: genera instancias y mide los tiempos de ambas variantes, escribiendo dos CSV (uno por variante).
- `regresion_BT.py`: ajusta los modelos teóricos a los datos experimentales y produce las gráficas.
- `Makefile`: el mismo de la sección de AR, compila también las dos variantes de *backtracking* (`solucionBT` y `solucionBT_noPoda`).

3.4. Estudio experimental del tiempo de ejecución

Comparamos las dos variantes (con poda y sin poda) sobre instancias aleatorias generadas por `tiempos_BT.cpp`. Como cada variante opera en un rango de tamaños muy distinto, preparamos dos modelos de prueba:

- **Sin poda:** fijamos $K = 2$ modelos por prenda y un presupuesto $M = 200 \cdot C$, para que el árbol se despliegue completo. Tamaños $C = 2, 4, \dots, 28$ (14 puntos), con dos generadores (uniforme en $[10, 200]$ y adversario en $[1, 50]$). Mediana de 51 repeticiones.
- **Con poda y memoización:** fijamos $K = 20$ (el máximo del enunciado) y $M = 0,85$ veces la suma de los precios máximos. Es el escenario más exigente para las podas, porque muchas combinaciones siguen siendo factibles. Tamaños $C \in \{5, 10, 15, 20, 25, 30, 35, 40, 50, 60, 70, 80, 90, 100, 120, 140, 160, 180, 200\}$ (19 puntos). Mediana de 201 repeticiones.

Las medidas obtenidas se recogen en las dos tablas siguientes:

Sin poda ($K = 2$)

C	Tiempo (ms)	C	Tiempo (ms)
2	0.000200	16	0.302900
4	0.000300	18	1.219200
6	0.000500	20	4.855000
8	0.002100	22	19.712200
10	0.007600	24	86.575500
12	0.026800	26	401.191900
14	0.081100	28	1768.160600

Con poda y memoización ($K = 20$, presupuesto ajustado)

C	Tiempo (ms)	C	Tiempo (ms)	C	Tiempo (ms)
5	0.039600	40	4.134800	100	25.404900
10	0.230900	50	6.407700	120	25.368100
15	0.555700	60	9.280300	140	32.262000
20	0.986500	70	12.543400	160	50.724300
25	1.592700	80	16.334600	180	64.335100
30	2.298700	90	20.504200	200	75.240400
35	3.166900				

La diferencia es enorme, pues sin poda, $C = 28$ ya tarda más de 1,7 segundos y con poda y memoización, $C = 200$ con $K = 20$ se resuelve en unos 75 ms. Además, en la versión sin poda el tiempo se multiplica aproximadamente por 4 cada vez que C aumenta en 2, justo el factor que predice $\Theta(2^C)$. La Figura 6 muestra cada serie por separado en escala lineal (con ejes propios, ya que los rangos son muy distintos), y la Figura 7 las superpone en escala semilogarítmica, donde se aprecia mejor la diferencia de varios órdenes de magnitud.

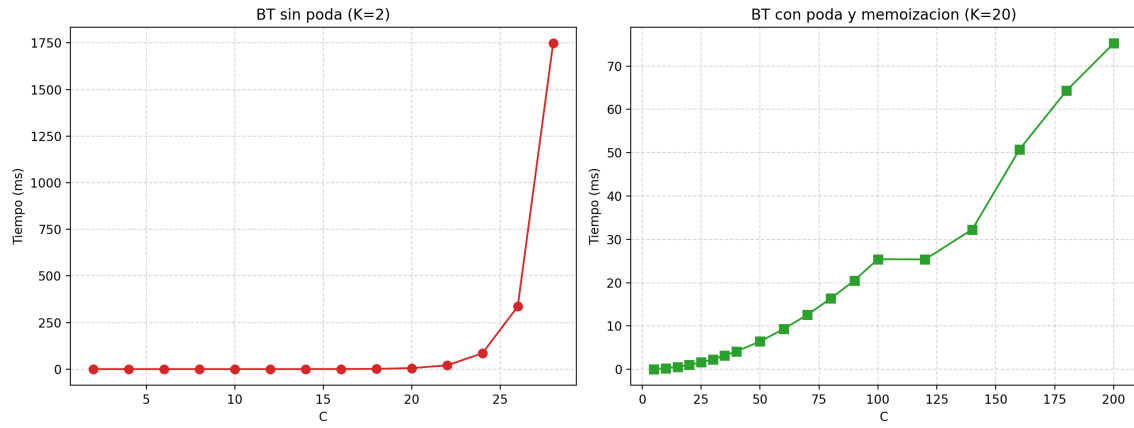


Figura 6: *Backtracking*: tiempos experimentales. A la izquierda la versión sin poda con $K = 2$ y $C \leq 28$. A la derecha la versión con poda y memoización con $K = 20$ y $C \leq 200$.

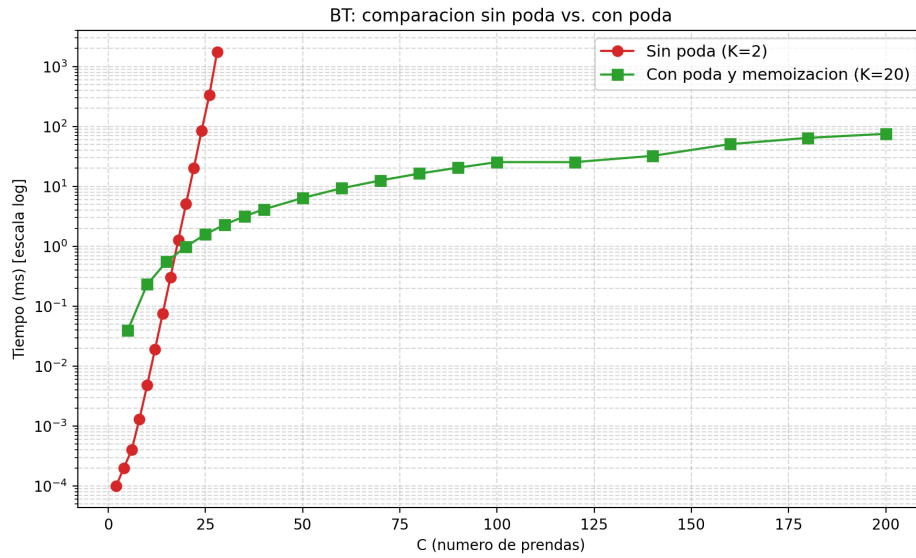


Figura 7: *Backtracking*: comparación de las dos variantes en escala semilogarítmica.

3.5. Contraste teórico-experimental.

Versión sin poda: ajuste a $\Theta(2^C)$

Ajustamos los 14 puntos al modelo $t(C) \approx a \cdot 2^C + b$ con `regresion_BT.py` y obtenemos:

- $a \approx 6,45 \times 10^{-6}$ ms.
- $b \approx -8,25$ ms.
- $R^2 = 0,9967$.

El ajuste es muy bueno y confirma la cota $\mathcal{O}(K^n)$ con $K = 2$ pues sin podas el algoritmo recorre prácticamente todo el árbol y crece de forma exponencial (Figura 8).

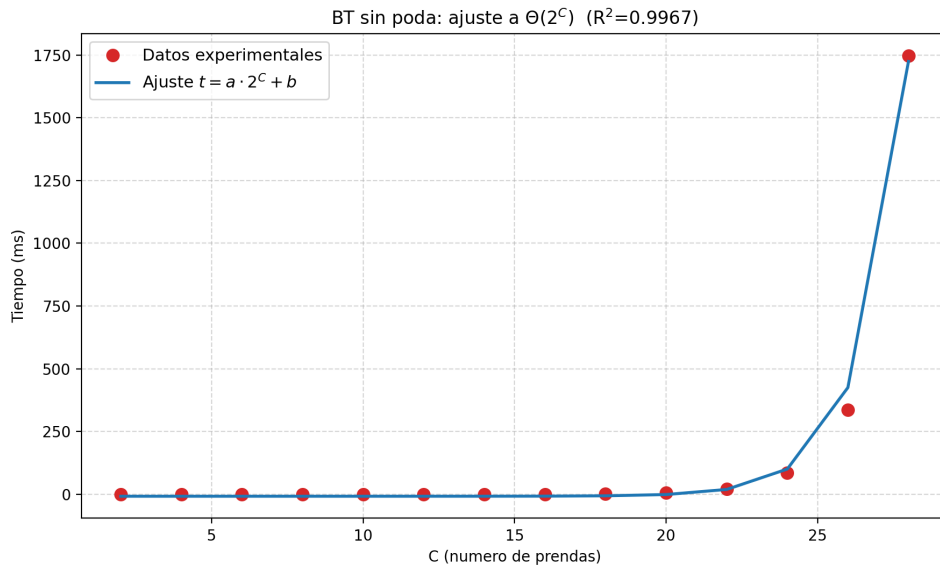


Figura 8: *Backtracking* sin poda: tiempos y ajuste $t \approx a \cdot 2^C + b$. $R^2 = 0,9967$.

Versión con poda y memoización, ajuste cuadrático.

En este experimento $K = 20$ es constante y M crece proporcional a C (el 85 % de la suma de los precios máximos), así que la cota $\mathcal{O}(n \cdot M \cdot K)$ se reduce a $\mathcal{O}(C^2)$. Por eso ajustamos al modelo $t(C) \approx a \cdot C^2 + b$. Sobre los 19 puntos:

- $a \approx 1,88 \times 10^{-3}$ ms.
- $b \approx 1,45$ ms.
- $R^2 = 0,9873$.

El R^2 es alto y los datos siguen claramente el orden cuadrático, incluso con presupuesto ajustado (que es donde las podas por optimalidad y memoización son menos efectivas). La Figura 9 muestra el ajuste.

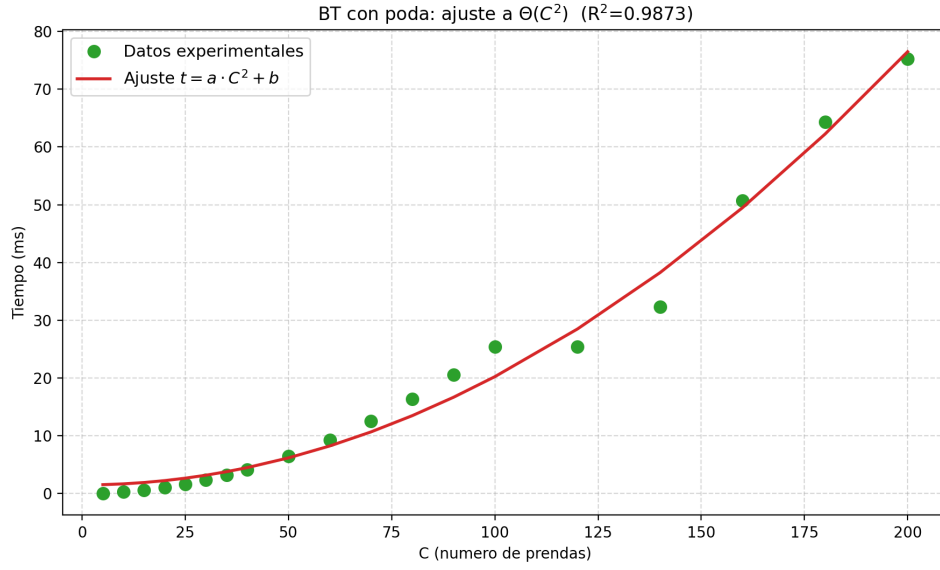


Figura 9: *Backtracking* con poda y memoización: tiempos y ajuste $t \approx a \cdot C^2 + b$. $R^2 = 0,9873$.

El experimento confirma la diferencia asintótica entre las dos versiones, ya que sin poda se ajusta a $\Theta(2^C)$ ($R^2 = 0,9967$) y con poda a $\Theta(C^2)$ ($R^2 = 0,9873$). En la práctica, con un presupuesto temporal de unos 100 ms la versión sin poda apenas llega a $C = 24$ con $K = 2$, mientras que la versión con poda alcanza $C = 200$ con $K = 20$. Las cuatro podas juntas (especialmente la memoización) son lo que nos permitió pasar de *Time Limit Exceeded* a *Accepted* en Mooshak.

4. Instrucciones de compilación y ejecución

En esta sección recopilamos los pasos necesarios para reproducir tanto los envíos al juez en línea como el estudio experimental. Todos los comandos se ejecutan desde el directorio raíz del proyecto (P2_AED2/). Hemos usado `g++ 15.2.0 (MSYS2)` y `Python 3.13` con los paquetes `numpy`, `matplotlib` y `scipy`.

4.1. Avance Rápido

Compilación y ejecución de la solución (Mooshak)

```
g++ -O2 -std=c++17 solucionAR.cpp -o solucionAR
./solucionAR < entrada.txt
```

La cabecera `solucionAR.h` debe estar en el mismo directorio. `entrada.txt` contiene los casos de prueba en el formato definido en el enunciado.

Tests unitarios

```
g++ -O2 -std=c++17 tests_unitarios_AR.cpp -o tests_AR
./tests_AR
```

Si superamos todos los tests, se imprime la línea `Todos los tests del AR superados.`

Estudio experimental y regresión

```
g++ -O2 -std=c++17 tiempos_AR.cpp -o tiempos_AR
./tiempos_AR > resultados_tiempos_AR.csv
python regresion_AR.py
```

El primer comando genera el CSV con la mediana de tiempos para los 19 tamaños $N \in \{4, 6, 8, \dots, 2048\}$ descritos en el estudio experimental. El script de Python lee dicho CSV, ajusta el modelo $t \approx a \cdot n^2 \log_2 n + b$ y genera la gráfica `ajuste_AR.png`.

4.2. Backtracking

Compilación y ejecución de la solución (Mooshak)

```
g++ -O2 -std=c++17 solucionBacktracking.cpp -o solucionBT
./solucionBT < entrada.txt
```

La versión sin poda (que usamos solo como referencia experimental) se compila de forma análoga:

```
g++ -O2 -std=c++17 solucionBacktracking_noPoda.cpp -o solucionBT_noPoda
```

Tests unitarios

```
g++ -O2 -std=c++17 tests_unitarios_BT.cpp -o tests_BT
./tests_BT
```

Estudio experimental y regresión

```
g++ -O2 -std=c++17 tiempos_BT.cpp -o tiempos_BT
./tiempos_BT > resultados_tiempos.csv
python regresion_BT.py
```

4.3. Gráficas adicionales

El script `graficas_extra.py` produce las gráficas comparativas (escala lineal, log-log y semilogarítmica) a partir de los dos ficheros CSV generados:

```
python graficas_extra.py
```

4.4. Compilación con make

Para no tener que invocar a mano cada `g++`, hemos añadido un **Makefile** en la raíz del proyecto que compila exclusivamente las soluciones en C++ (los binarios que se envían a Mooshak y la variante sin poda usada como referencia experimental). Las baterías de medición y los *scripts* de regresión y gráficas se ejecutan aparte, como se ha indicado en las secciones anteriores. Los *targets* disponibles son:

```
make          # solucionAR, solucionBT y solucionBT_noPoda
make ar       # solo solucionAR
make bt       # solucionBT y solucionBT_noPoda
make run-ar   # ./solucionAR < entrada.txt
make run-bt   # ./solucionBT < entrada.txt
make clean    # elimina los binarios
```

Internamente usa `CXX=g++` y `CXXFLAGS=-O2 -std=c++17 -Wall`, las mismas opciones que en los comandos anteriores.

5. Uso de la inteligencia artificial

A lo largo de la práctica hemos utilizado herramientas de IA generativa (principalmente *Claude* y *ChatGPT*) como apoyo en tareas concretas, siempre revisando y verificando los resultados antes de incorporarlos. Detallamos a continuación los usos:

- **Redacción y revisión del informe:** hemos pedido revisiones de estilo y ortografía sobre el `.tex`, así como sugerencias para corregir voz pasiva, simplificar los análisis experimentales y unificar el uso de la primera persona del plural. La estructura, los datos numéricos, las conclusiones y las decisiones de diseño son nuestros; la IA solo ha servido como corrector de estilo.
- **Formato L^AT_EX:** resolución de pequeños problemas tipográficos (ligadura » en `\texttt`, ajuste de tablas, entornos `algorithm` y `lstlisting`). El esquema general del documento se realizó manualmente.
- **Depuración puntual de C++:** consultas sobre errores de compilación detectados por Mooshak (por ejemplo, el *casting* olvidado de `S.size()` a `int`) y dudas sobre el comportamiento de `std::sort` con comparadores personalizados. El diseño de los algoritmos, las podas y la elección de estructuras de datos son nuestros.
- **Scripts auxiliares en Python:** apoyo en el código de `regresion_AR.py`, `regresion_BT.py` y `graficas_extra.py` (uso de `numpy`, `scipy.optimize.curve_fit` y `matplotlib`), revisando manualmente los ajustes y los valores de R^2 obtenidos.
- **Generación de la cabecera `solucionAR.h`:** a partir del `solucionAR.cpp` ya escrito por nosotros, pedimos que se extrajera la interfaz pública (structs y prototipos) siguiendo el estilo de `solucionBacktracking.h`.
- **Validación de pseudocódigo:** hemos contrastado con la IA que el pseudocódigo del informe coincide con el código C++ real y con el esquema teórico visto en clase.
- **Generación del archivo `Makefile`.**

En ningún caso hemos delegado en la IA la toma de decisiones sobre el diseño de los algoritmos ni la interpretación de los resultados experimentales; las medidas, los ajustes y las conclusiones se han comprobado por nosotros. Toda salida sugerida por estas herramientas se ha leído y, cuando ha sido necesario, modificado antes de incluirla en el proyecto.

6. Conclusiones

6.1. Valoraciones personales

Esta práctica nos ha servido para aplicar de forma directa dos esquemas que hasta ahora habíamos visto sobre todo en teoría.

Por una parte, el problema de *Organización en Clase*, resuelto con un voraz, es engañosamente sencillo, ya que la idea es inmediata, pero hay detalles que importan, como evitar borrar elementos de la lista de candidatos para no incurrir en un coste lineal innecesario. El análisis teórico nos dio una cota $\Theta(n^2 \log n)$ dominada por el `sort` inicial, y al medirlo en la práctica coincidía bien con la teoría.

Por la otra parte, en *Nos Vamos de Boda*, resuelto por *backtracking*, lo más importante han sido las podas. Las primeras versiones que enviamos a Mooshak compilaban y daban resultado correcto en casos pequeños, pero superaban el límite de tiempo en cuanto el árbol crecía. Lo que terminó funcionando fue añadir las cuatro condiciones de poda juntas, y especialmente la matriz de visitados... pasamos de un coste exponencial a uno acotado por $\mathcal{O}(n \cdot M \cdot K)$, y eso es lo que se ve en la gráfica del ajuste cuadrático.

También nos ha resultado útil el proceso de medir tiempos y ajustar los modelos teóricos a los datos reales. Antes lo veíamos como algo más bien teórico, pero al hacerlo nosotros queda mucho más claro cuándo una cota describe bien el comportamiento del programa y cuándo hay que pensar mejor el escenario de prueba.

6.2. Estimación de tiempo invertido

- **Adriana Piñero Iglesias:** aproximadamente 25 horas.
- **Pablo Pérez Tudela:** aproximadamente 25 horas.